

Contenido

- 1. Variables globales encontradas 2
- 2. Dependencias entre modulos 2
- 3. Duplicacion de codigo 3
- 4. Bare except y manejo de errores 3
- 5. Archivos *_config.py no usados 4
- 6. Consolidado: pasos y riesgos 4

Analisis Tecnico del Proyecto Refactoring

Variables globales - Dependencias - Duplicacion - Errores - Configuracion | 2026-09-22

Alcance: api_movies.py (187 lineas) + main.py (334 lineas). Fase: analisis, sin cambios de logica aplicados.

1. Variables globales encontradas

Regla usada: solo cuenta lo definido a nivel de modulo con estado compartido. Todas las definiciones estan en api_movies.py; main.py no define ninguna global propia.

Variable (archivo:linea)	Tipo / valor	Uso	Mantener global?
API_KEY_OMDB (api_movies.py:5)	str = "trilogy"	URLs OMDB (45, 56)	Si: configuracion
BASE_URL_OMDB (api_movies.py:7)	str URL	URLs OMDB (45, 56)	Si
BASE_URL_TVMAZE (api_movies.py:9)	str URL	URLs TVMaze (71, 79)	Si
API_KEY_TMDB (:6) BASE_URL_TMDB (:8)	str vacia / URL	Sin uso en ningun archivo	No: eliminar
USUARIO_LOGUEADO (api_movies.py:12)	None	Sin lectura ni escritura	No: global muerto
PELICULAS_FAVORITAS (api_movies.py:13)	list []	Fav. + main.py 198-208	Temporalmente si
HISTORIAL_BUSQUEDAS (api_movies.py:14)	list []	Hist. + main.py 219-222	Temporalmente si
CACHE_PELICULAS (:15) CACHE_SERIES (:16)	dict {}	Solo interno api_movies	Si: cache idiomática
CONFIG (api_movies.py:17-22)	dict 4 claves	Ambos; main.py 260-272	Temporalmente si
main.py: sin globales propios	---	Consume 3 via import *	---

1.1 Lo que NO es global

- Locales de funcion (titulo, pelicula, data, url, indice, stats...), listas creadas dentro de obtener_peliculas_populares (84-90) y buscar_por_genero (94-101), funciones e imports.

1.2 Matiz sobre global innecesario

- Solo hace falta 'global' si se REASIGNA el nombre. Innecesario en buscar_pelicula (38), buscar_series (65), agregar_a_favoritas (112), eliminar (128) y agregar_historial (138): solo mutan. Necesario en limpiar_historial (146) e importar_de_json (179): hacen 'lista = [...]'.

1.3 Recomendacion y riesgo

- Dejar constantes; eliminar USUARIO_LOGUEADO, API_KEY_TMDB, BASE_URL_TMDB y CONFIG[max_retries] (muertas); imports explicitos; quitar 'global' innecesarios.
- Riesgo: reasignar listas con 'import *' deja a main.py con la referencia vieja; renombrar claves de CONFIG rompe hacer_request y funcion_configuracion.

2. Dependencias entre módulos

Dirección única: main.py -> api_movies.py (vía 'from api_movies import *', main.py:7). api_movies.py no importa nada propio (solo requests + json). Ningún otro .py importa estos módulos. app.py es un duplicado monolítico huérfano (ver 3.5).

```
main.py --import *--> api_movies.py --requests/json--> OMDb + TVMaze + .json
```

2.1 Uso función por función (main -> api_movies)

- función_buscar_pelicula: buscar_pelicula + agregar_al_historial + agregar_a_favoritas.
- función_buscar_actor: buscar_peliculas_por_actor + buscar_pelicula. función_buscar_series: buscar_series + obtener_detalle_serie.
- Resto: obtener_peliculas_populares, buscar_por_genero, eliminar_de_favoritas, limpiar_historial, obtener_estadísticas, exportar/importar_json.
- Lectura directa de globales: PELICULAS_FAVORITAS (198-208), HISTORIAL_BUSQUEDAS (219-222), CONFIG (260-272). Interno: hacer_request() sirve a 4 búsquedas.

2.2 Riesgo principal

- Quitar 'import *' sin explícitos rompe todo (NameError); renombrar una función en api_movies rompe main en cadena; cambiar hacer_request afecta a 4 funciones.

3. Duplicación de código

3.1 Lógica de negocio (api_movies.py)

- D1 (36-52 vs 63-75): cache idéntica en buscar_pelicula/buscar_series. D2 (115-132, 152-157): recorridos manuales + conteo 'total = total + 1' en vez de len(). D3 (84-101): listas {titulo, año, rating} en populares y por_genero.

3.2 Interfaz (main.py)

- D4 (35-78): 9 bloques try/except por campo. D5 (101, 159, 199, 220): 4 bucles 'while i' en vez de for/enumerate. D6 (11 sitios): literal input('Presione Enter...') -> candidata a pausa(). D7 (110-193): esqueleto input->buscar->mostrar->pausa en 5 funciones.

3.3 Nivel archivo

- D8: main.py (334 lin) vs app.py (255 lin): dos implementaciones paralelas con otros nombres (buscar_pelicula_omdb, print_sep...). Decidir un ganador; no mantener ambos.

3.4 Propuestas (sin aplicar)

- len()/any(), helper mostrar_campo o .get(), for+enumerate, pausa() de 1 línea. D1/D7 pueden quedar duplicados (8 líneas): legibilidad antes que abstracción.

4. Bare except y manejo de errores

4.1 Bare except localizados

Sitio	Codigo	Problema
E1-E9 main.py 35-77	try/except: desnudo x9 campos (Title, Year, ...)	Tapa TypeError y KeyboardInterrupt
E10 main.py 251-253	except: generico al importar	Sin diagnostico del fallo real
E11 (OK) main.py 327-333	except KeyboardInterrupt + Exception as e	Correcto: dejar como referencia

4.2 Errores por ausencia de control

Sitio	Que falta	Riesgo
A1 - api 24-34	requests.get + .json() sin try ni status	Caída sin red o API caída
A2 - api 45-79	URL por concatenacion sin validar	Titulos con espacios o & rompen query
A3/A4 - api 164-187	open/json sin try; importar sin validar	OSError o JSON corrupto
A5 - main 272	int(input timeout) sin try	ValueError cierra el programa
A6 - main 95	summary None + [:200] sin guarda	TypeError si API devuelve null
A7 - main 103-222	accesos ["clave"] sin .get()	KeyError tras importar datos

4.3 Propuestas (sin aplicar)

- E1-E9: pelicula.get('Title', 'N/A'). E10: except (FileNotFoundError, JSONDecodeError, OSError) as e. A1: try/except RequestException -> return None + comprobar None en llamantes. A5: try int/except ValueError.
- Riesgo: concretar excepciones revelara fallos hoy ocultos (deseado); hacer_request devolviendo None exige actualizar 4 llamantes. No tocar E11.

5. Archivos *_config.py no usados

- 88 archivos (de 118 .py): nucleo ~23 + familia cache/API ~65. Verificacion: 0 imports desde no-config (main/api_movies/app/utills/managers). Funciones load_*/get_* nunca llamadas.
- Patron repetido 88 veces: *_CONFIG_FILE str + *_config dict + load/save/get/set; time/datetime importados sin uso; ~176 globales muertos. Solapan con CONFIG real sin conectarse.
- Propuesta: archivar en legacy_config/ (reversible), conservar maximo cache_config.py y api_config.py como referencia. Riesgo de borrar hoy: casi nulo; riesgo de mantener: ruido y .json huérfanos.

6. Consolidado: pasos y riesgos

- Paso 1 (seguro): borrar muertas, quitar 'global' innecesarios, imports explicitos. Paso 2: .get() y excepciones concretas. Paso 3: len/for/pausa, app.py y *_config.py a legacy.
- Nada aplicado aun: este documento es analisis. Codigo intacto hasta revision y orden explicita.